

Precision-AMR: Precisão de Ponto Flutuante Espacialmente Adaptativa para Migração Reversa no Tempo Baseada em EDPs

Um novo eixo de adaptividade numérica para solucionadores adjuntos da equação da onda, desenvolvido em hardware NVIDIA dedicado e validado em arquiteturas heterogêneas de GPUs NVIDIA e AMD no supercomputador Santos Dumont (LNCC)

Pesquisador Principal: Prof. Sandro Rigo

Cargo/Função: Professor Associado, Instituto de Computação

Universidade: Universidade Estadual de Campinas (UNICAMP)

Colaborador 1: Thiago Maltempi — Pesquisador de Pós-Graduação (Doutorando), Instituto de Computação, Universidade Estadual de Campinas

Colaborador 2: Prof. Hervé Yviquel — Professor Assistente, Instituto de Computação, Universidade Estadual de Campinas

1 Enunciado do Problema

O hardware de GPU vem se voltando cada vez mais para a baixa precisão em prol de cargas de trabalho de IA (FP8/BF16), expondo novas hierarquias de memória, tensor cores e bibliotecas matemáticas de precisão mista. Ao mesmo tempo, simulações científicas baseadas em equações diferenciais parciais (EDPs) continuam a exigir precisão FP64/FP32 para permanecer confiáveis. A Migração Reversa no Tempo (Reverse Time Migration, RTM), método de imageamento sísmico que resolve a equação da onda por diferenças finitas ao longo de milhares de passos de tempo, é um exemplo canônico dessa tensão: geofísicos usam a imagem migrada para orientar decisões de perfuração de alto custo, de modo que a fidelidade numérica é essencial. A RTM também é intensiva em memória, exigindo centenas de gigabytes de estado do campo de onda e dependendo de checkpointing para trocar memória por recomputação em sua estrutura adjunta de propagação direta/reversa.

Nosso trabalho anterior atacou o principal gargalo dessa estrutura, isto é, a transferência de checkpoints entre host e GPU via PCIe, com a combinação do conceito de prefetching com a compressão de dados. Essa ideia foi materializada como a biblioteca GPUZIP [3, 4], que comprime os checkpoints na GPU e os pré-busca de forma proativa, reduzindo o volume de transferência host–GPU em até $80\times$ e alcançando ganhos de desempenho ponta a ponta de até $5,1\times$ na V100, preservando a qualidade da imagem ($SSIM \approx 1$, erro relativo próximo de zero). Separadamente, já se demonstrou que a aritmética de precisão reduzida é viável para modelagem sísmica, RTM e FWI: Fabien-Ouellet [1] constatou que uma implementação estável em meia precisão (FP16) da equação de onda elástica é alcançável por meio de escalonamento (*scaling*) apropriado, que o erro resultante no sismograma é uma fração desprezível da energia sísmica total e majoritariamente incoerente com as fases sísmicas, e que esse ruído não prejudica a qualidade das imagens de FWI ou RTM. Entretanto, essa redução é aplicada de forma uniforme em todo o domínio computacional. Gao e Harms [2] identificaram um risco específico dessa abordagem: a integração no tempo em meia precisão de forma ingênua acumula erro de arredondamento por meio de uma somatória recursiva disfarçada na regra de atualização, degradando a qualidade da solução, sendo a somatória compensada (de Kahan) um remédio eficaz.

Ao mesmo tempo, o refinamento adaptativo de malha (Adaptive Mesh Refinement, AMR) tem sido explorado na simulação de ondas sísmicas e, mais recentemente, explicitamente em fluxos de trabalho de FWI e RTM, para evitar o desperdício inerente a uma malha uniforme: os erros de dispersão e dissipação exigem espaçamento de malha proporcional ao menor comprimento de onda, de modo que uma malha uniforme é forçada a usar o espaçamento mais fino em toda parte, mesmo onde o campo de velocidade local toleraria um espaçamento bem mais grosseiro. O AMR resolve isso refinando o Δx localmente. Não existe, porém, esquema comparável e fisicamente motivado para a precisão de ponto flutuante: os trabalhos existentes de precisão mista em imageamento sísmico são globais e uniformes (como acima), ou adaptativos apenas no nível de uma operação individual de álgebra linear (GEMM de precisão mista, refinamento iterativo, emulação pelo esquema de Ozaki), sem estar vinculados a uma decomposição espacial do próprio campo de onda. Nenhum desses trabalhos varia a precisão de ponto flutuante segundo critérios espaciais fisicamente motivados. Esse é um terceiro eixo de adaptabilidade que permanece, portanto, praticamente inexplorado.

Este projeto propõe preencher essa lacuna com o Precision-AMR (P-AMR): um esquema espacialmente estruturado em blocos, análogo de precisão adaptativa ao AMR, que mantém fixa a resolução da malha e, em vez disso, varia o formato de ponto flutuante atribuído a cada bloco (*tile*) do domínio, com base em critérios de tolerância a erro fisicamente motivados (proximidade a bordas absorventes, densidade de energia do campo de onda, magnitude da sensibilidade adjunta, e uma faixa dependente do tempo

que segue a frente de onda em propagação) que preveem se o erro de arredondamento introduzido naquele bloco pode alcançar a condição de imageamento ou o gradiente do FWI. Trata-se de um problema estruturalmente diferente do AMR baseado em malha. No AMR, o artefato característico é uma reflexão localizada na interface grossa-fina da malha, porque o refinamento altera a própria discretização. No P-AMR, nenhuma fronteira de discretização é introduzida; em vez disso, um nível espacialmente variável de ruído de arredondamento é injetado em uma EDP hiperbólica, e esse ruído se propaga junto com a onda. Os critérios de marcação (*tagging*), portanto, não podem se basear apenas na suavidade local, como fazem os indicadores de refinamento do AMR; eles precisam, em vez disso, aproximar onde um erro, uma vez injetado, decairá, será absorvido ou deixará de alcançar os receptores ou o kernel de sensibilidade antes do fim da simulação, e controlar o acúmulo de arredondamento ao longo dos milhares de passos de tempo típicos de RTM/FWI.

Como a RTM/FWI já exige uma decomposição do domínio em blocos/*tiles* para o checkpointing, o P-AMR reaproveita essa mesma decomposição como granularidade de marcação, unificando o orçamento de precisão e o orçamento de compressão com perdas do checkpointing diferencial em uma única meta de precisão por bloco, conjuntamente ajustada. O GEMM de precisão mista, o refinamento iterativo do cuSOLVER e a emulação de FP64 pelo esquema de Ozaki continuam essenciais como técnicas de suporte, mas passam a ser aplicados seletivamente, apenas onde os critérios do P-AMR indicam necessidade — a forma natural de explorar GPUs orientadas primariamente para IA, como a RTX PRO 6000 Blackwell, que oferecem vazão nativa de FP64 desprezível.

Como os critérios de marcação são motivados pela física da propagação de ondas e pela sensibilidade adjunta, e não pelas unidades aritméticas de um fabricante específico, tratamos sua portabilidade entre arquiteturas como uma questão explícita, e não como uma suposição. Nosso grupo de pesquisa tem acesso ao supercomputador Santos Dumont (LNCC/SINAPAD), que inclui aceleradores AMD Instinct MI300A ao lado de diversas gerações NVIDIA (Volta V100, Hopper H100, Grace-Hopper GH200). Isso nos permite testar essa questão diretamente, em vez de apenas afirmá-la, e é o que confere ao Precision-AMR sua contribuição potencial para a área: um eixo de adaptividade numérica novo e portátil entre fabricantes, e não apenas uma otimização pontual para uma única arquitetura de GPU.

2 Plataformas Computacionais

HPC SDK (nvc++/nvfortran, bibliotecas matemáticas CUDA-X, Nsight Systems/Compute). Experiência: a equipe usa CUDA e Nsight Systems rotineiramente. Adoção: migrar a compilação do Awave-3D de clang para nvc++ e validar a paridade numérica em relação às linhas de base (*baselines*) existentes. O Nsight Compute fornece análise *roofline* em

nível de kernel, agora aplicada por bloco espacial, para identificar quais blocos são limitados pela largura de banda de memória versus limitados por computação, e para delimitar objetivamente onde um bloco de menor precisão realmente compensa.

RTX PRO 6000 Blackwell (hardware dedicado, solicitado). Os tensor cores de 5ª geração dessa GPU, com suporte a FP8/FP4 e vazão nativa de FP64 desprezível, a tornam a plataforma central para o P-AMR: como a emulação de FP64 em todo o domínio seria cara em toda parte, a questão científica que este projeto responde é precisamente quais blocos precisam dela e quais não precisam. Sua memória de 96 GB GDDR7 com ECC também sustenta o desenvolvimento local; por se tratar de hardware dedicado e sem fila de espera, permite depuração iterativa rápida, algo para o qual um sistema de lote compartilhado não é bem adequado.

Santos Dumont (LNCC/SINAPAD, supercomputador nacional Tier-0, Petrópolis-RJ). Fornece o ambiente de validação em larga escala. Sua partição BullSequana X1000 inclui 94 nós com 4× GPUs NVIDIA Volta V100 cada — compatível com o hardware usado nas linhas de base originais do GPUZIP — além de um nó de IA dedicado com 8× V100 de 16GB via NVLink. Sua partição mais recente, BullSequana XH3000, acrescenta 62 nós com 4× NVIDIA Hopper H100 SXM de 80GB via NVLink, 36 nós com superchips NVIDIA Grace-Hopper GH200, e 18 nós com 2× APUs AMD Instinct MI300A (núcleos de GPU CDNA3, 128 GB de HBM3 cada). Essa heterogeneidade nos permite gerar a referência de verdade (*ground truth*) em FP64 e executar em escala a varredura completa dos critérios do P-AMR em hardware NVIDIA, além de testar separadamente se a metodologia de marcação e a abordagem de tiling em blocos generalizam para uma arquitetura não NVIDIA.

nvCOMP / Bitcomp (compressor com perdas executado na GPU). Experiência: integrado e validado no GPUZIP v2.0, onde o Bitcomp apresentou o melhor equilíbrio entre precisão e compressão na V100. Adoção aqui: o mecanismo de armazenamento do checkpointing diferencial, agora indexado pelos mesmos identificadores de bloco usados na marcação de precisão, de modo que cada bloco carrega um único orçamento de erro conjuntamente ajustado para precisão e compressão.

cuBLAS (extensões de precisão mista) e cuSOLVER (refinamento iterativo). Ambos vêm incluídos no HPC SDK. Experiência: a equipe já utilizou rotinas BLAS/LAPACK em trabalhos anteriores de HPC. Adoção: aplicados apenas nos blocos que o P-AMR sinaliza como contendo subkernels densos que necessitam de precisão além da baixa precisão nativa (computação em FP32 com acumulação em FP64, e refinamento iterativo do cuSOLVER).

Emulação de FP64 pelo esquema de Ozaki via tensor cores de baixa precisão. Adoção: reservada ao subconjunto de blocos que os critérios de sensibilidade do P-AMR marcam como críticos para a precisão (próximos à frente de onda, com alta magnitude de sensibilidade adjunta), em vez de aplicada uniformemente — testando diretamente se a emulação seletiva é mais barata do que emular em todo o domínio, preservando a fidelidade da imagem. Trabalhos recentes demonstraram ganhos de até $2,3\times$ em relação ao DGEMM nativo em FP64 na RTX PRO 6000 Blackwell [5].

cuFFT. Experiência: utilizado em trabalhos anteriores com operadores espectrais. Adoção: avaliado como alternativa aos estênceis de diferenças finitas (FDM) para a atualização do campo de onda dentro de blocos selecionados, marcados por precisão, caracterizando o compromisso entre largura de banda de memória e computação em relação ao caminho padrão de diferenças finitas.

AMD Instinct MI300A (CDNA3, Santos Dumont, exploratório). Como atividade adicional de validação, avaliamos quanto do framework P-AMR — a decomposição em blocos/tiles, os quatro critérios de marcação e o orçamento conjunto de erro de precisão e compressão — se transfere para o ecossistema ROCm/HIP da AMD (rocBLAS, hipBLAS e os próprios caminhos de matriz de precisão reduzida da AMD), sem assumir paridade de recursos com a emulação do esquema de Ozaki, específica da CUDA, ou com o caminho de compressão nvCOMP/Bitcomp. O objetivo é determinar se o Precision-AMR é uma metodologia portátil ou uma otimização específica da NVIDIA.

3 Conjunto de Dados e Modelo

- Tamanho: $\approx$ XTB no total (a confirmar junto ao REDU). Os conjuntos de checkpoints chegaram a $\approx$ 330 GB de RAM do host na V100; um único checkpoint do Marmousi3D tem $\approx$ 500 MB.
- Tipo de dado: modelos de velocidade sísmica e traços (malhas numéricas).
- Origem: Salt (SEG/EAGE) e Marmousi3D são conjuntos de dados abertos; o conjunto de dados Large é interno (*in-house*).
- Confidencialidade: não confidencial, sem dados pessoais, sem NDA.
- Parâmetros de modelo de IA: não aplicável (solucionador numérico de EDPs).
- Local de armazenamento: Universidade Estadual de Campinas (UNICAMP), Campinas, Brasil; preparado para envio aos sistemas de arquivos paralelos do Santos Dumont para as execuções em larga escala.

4 Métodos

4.1 Fase 1 — Infraestrutura e Linha de Base (M1–M2)

Portar o Awave-3D e o GPUZIP para o HPC SDK (nvc++), reintegrar o nvCOMP/Bitcomp e reproduzir as linhas de base em V100 na partição V100 do Santos Dumont, compatível com o hardware original do GPUZIP.

- Estabelecer uma decomposição do domínio em blocos/*tiles*, estendendo a estrutura de blocos de checkpoint já existente para que também possa carregar uma marca (*tag*) de precisão por bloco.
- Perfilar (*profile*) todos os kernels com Nsight Systems e Nsight Compute, tanto na RTX PRO 6000 quanto no Santos Dumont, confirmando que os estênceis de diferenças finitas (FDM) são limitados pela largura de banda de memória na granularidade de bloco antes de qualquer alteração de precisão ser tentada.

4.2 Fase 2 — Precision-AMR: Critérios, Marcação e Validação (M2–M6)

Esta é a fase científica central, executada em escala na partição H100 do Santos Dumont. Projetamos, implementamos e validamos quatro critérios candidatos de marcação para o zoneamento espacial de precisão:

- **Critério de borda:** blocos dentro ou adjacentes à camada absorvente PML são marcados por padrão para precisão reduzida, já que sua contribuição já está sendo amortecida.
- **Critério de energia:** os blocos são marcados por um limiar sobre a densidade de energia local do campo de onda, de modo que regiões de baixa amplitude (antes da chegada da frente de onda, ou após o espalhamento geométrico ter atenuado o sinal) tolerem menor precisão.
- **Critério de sensibilidade (FWI):** os blocos são marcados pela magnitude do kernel de sensibilidade adjunta local, reaproveitando o próprio cálculo de gradiente como sinal de marcação, em analogia direta a um estimador de erro de truncamento de Berger–Oliger no AMR clássico.
- **Critério de esteira da frente de onda:** uma faixa dependente do tempo, imediatamente atrás da frente de onda em propagação, é marcada para precisão reduzida, enquanto a borda dianteira — onde a precisão de fase rege a condição de imageamento — mantém precisão plena.

Para cada critério, implementamos a somatória compensada (de Kahan) na atualização de integração no tempo dos blocos de baixa precisão, para controlar o acúmulo de

arredondamento ao longo de toda a simulação, seguindo o mecanismo identificado por Gao e Harms [2]. Testamos explicitamente a ocorrência de um artefato de “interface de precisão” — análogo às reflexões de interface grossa-fina do AMR baseado em malha — por meio de análise de resíduo nas fronteiras entre blocos de precisões diferentes. Por fim, cada bloco recebe conjuntamente um formato de ponto flutuante e uma política de compressão de checkpoint (*snapshot* completo versus delta comprimido), com a precisão combinada medida por PSNR, SSIM e erro relativo médio em relação à linha de base em FP64, tanto no campo de onda bruto quanto na imagem migrada final. Esta fase absorve o checkpointing diferencial, que passa a ser o mecanismo de armazenamento para os blocos marcados pelo P-AMR, em vez de um entregável separado.

4.3 Fase 3 — Precision-AMR sem FP64 Nativo (M7–M10)

Os critérios de marcação validados na Fase 2 são aplicados na RTX PRO 6000, hardware com vazão nativa de FP64 desprezível. Por bloco, o *pipeline* seleciona entre execução nativa de baixa precisão em tensor cores, GEMM de precisão mista via cuBLAS (computação em FP32 com acumulação em FP64) ou refinamento iterativo do cuSOLVER para subkernels densos, e emulação de FP64 pelo esquema de Ozaki para blocos sinalizados como críticos para a precisão.

- Isso operacionaliza diretamente a motivação de explorar GPUs voltadas primariamente para IA que carecem de FP64 nativo: a emulação, cara, é concentrada apenas onde os critérios do P-AMR indicam necessidade.
- Quantificamos o compromisso entre ganho de desempenho e precisão em relação a uma linha de base com Ozaki uniforme (emulando FP64 em todo o domínio), para determinar quanto o *tiling* seletivo realmente economiza.
- Todos os erros são delimitados conjuntamente (interação compressão $\times$ precisão) em relação à linha de base em FP64, usando PSNR, SSIM e erro relativo médio, nos mesmos três conjuntos de dados e três algoritmos de checkpointing (Revolve, zCut, Uniform) usados ao longo de todo o projeto.
- Atividade exploratória entre arquiteturas: a mesma lógica de marcação e *tiling* é portada para os nós AMD Instinct MI300A do Santos Dumont via ROCm/HIP, para testar se os critérios e o *framework* de orçamento de erro se sustentam em uma arquitetura CDNA3 sem acesso nativo à emulação pelo esquema de Ozaki ou ao nvCOMP/Bitcomp.

4.4 Fase 4 — Operadores Espectrais, Integração e Lançamento (M10–M12)

- Avaliar operadores de derivada pseudo-espectrais baseados em cuFFT como regra de atualização alternativa para blocos selecionados, marcados por precisão, como técnica secundária que complementa a contribuição central do P-AMR.
- Integrar o módulo Precision-AMR, o checkpointing diferencial e os caminhos de precisão mista em um lançamento unificado do GPUZIP v3.0.
- Avaliação de escalabilidade multi-nó no Santos Dumont (≤ 8 GPUs).
- Análise final, redação e submissão, com o Precision-AMR como contribuição científica central e o checkpointing diferencial, a álgebra linear de precisão mista e os operadores espectrais apresentados como técnicas de suporte.

5 Avaliação

Todos os experimentos abrangem três conjuntos de dados (Large, Marmousi3D, Salt), três algoritmos de checkpointing (Revolve, zCut, Uniform) e os quatro critérios de marcação do P-AMR (individualmente e em combinação). A perfilagem em nível de kernel usa o Nsight Compute (*roofline*, detalhamento de tráfego de memória); a perfilagem de linha do tempo usa o Nsight Systems. A implementação é em C/C++ com CUDA sob o HPC SDK, com OpenMP Cluster para escalabilidade multi-nó. A precisão é avaliada tanto no campo de onda quanto na imagem final de RTM, já que um *tiling* que preserve PSNR/SSIM do campo de onda ainda pode, em princípio, deslocar a imagem migrada se o erro for espacialmente correlacionado com refletores. Para a atividade entre arquiteturas, as mesmas métricas de precisão são calculadas na partição AMD MI300A do Santos Dumont e comparadas às linhas de base NVIDIA, para caracterizar quanto do ganho de desempenho e do compromisso de precisão alcançados independe da arquitetura.

6 Resultados Esperados

Cronograma (12 meses):

Mês	Atividade	Hardware	Fase
M1–M2	Portar Awave-3D + GPUZIP para o HPC SDK (nvc++); reproduzir linhas de base em V100; estabelecer decomposição em blocos/tiles; perfilagem <i>roofline</i> do Nsight Compute por bloco.	RTX PRO 6000; Santos Dumont (V100)	1

Mês	Atividade	Hardware	Fase
M2–M5	Projetar e implementar os critérios de marcação do Precision-AMR (proximidade à PML, densidade de energia, sensibilidade adjunta, faixa de esteira da frente de onda); somatória compensada; teste de artefato de interface de precisão.	Santos Dumont (H100)	2
M5–M6	Unificar o orçamento de precisão e compressão via checkpointing diferencial; escalabilidade multi-nó; lançamento de código aberto do GPUZIP v3.0-alpha; pacote de reprodutibilidade (REDU).	Santos Dumont (H100, multi-nó)	2
M7–M10	Aplicar o <i>tiling</i> do Precision-AMR sem FP64 nativo: GEMM de precisão mista via cuBLAS seletivo, refinamento iterativo do cuSOLVER e emulação de FP64 pelo esquema de Ozaki apenas nos blocos críticos para a precisão. Portabilidade exploratória do <i>framework</i> de marcação para o AMD Instinct MI300A via ROCm/HIP.	RTX PRO 6000; Santos Dumont (AMD MI300A, exploratório)	3
M10–M12	Avaliação de operadores espectrais cuFFT em blocos selecionados; lançamento unificado do GPUZIP v3.0; análise final, redação e submissão do artigo.	RTX PRO 6000; Santos Dumont	4

Resultados projetados: um lançamento de código aberto do GPUZIP v3.0 (licença MIT, GitHub) construído em torno de um módulo Precision-AMR, com checkpointing diferencial, caminhos de GEMM de precisão mista/esquema de Ozaki e operadores espectrais cuFFT integrados como componentes de suporte; uma publicação revisada por pares com o Precision-AMR como contribuição central (alvo: SBAC-PAD / Euro-Par / IJHPCA); e um pacote público de reprodutibilidade (conjuntos de dados, perfis do Nsight, métricas de qualidade) via REDU da UNICAMP.

Dependências: esta solicitação de hardware não está condicionada à combinação com outro financiamento. As execuções de validação em larga escala no supercomputador Santos Dumont (LNCC/SINAPAD) ocorrem por meio do próprio programa de alocação computacional do LNCC (SINAPAD), independentemente desta solicitação. O apoio institucional existente financia pessoal; nenhuma dessas fontes duplica a solicitação da RTX

PRO 6000.

7 Detalhes do Apoio ao Projeto

Solicitamos uma GPU RTX PRO 6000 Blackwell como hardware de desenvolvimento local dedicado. Sua abundante vazão de tensor cores em FP8/FP4 e sua vazão nativa de FP64 desprezível a tornam o alvo cientificamente relevante para o estudo de emulação seletiva central à Fase 3: ela força a questão de saber se o uso seletivo de emulação pelo esquema de Ozaki pelo Precision-AMR consegue igualar a emulação em todo o domínio a uma fração do custo. Por se tratar de hardware dedicado e sem fila de espera, também sustenta o desenvolvimento iterativo cotidiano para o qual um sistema de lote compartilhado não é bem adequado. A validação em larga escala — a varredura dos critérios do P-AMR entre conjuntos de dados e algoritmos de checkpointing, a geração da linha de base em FP64, a escalabilidade multi-GPU e o teste de portabilidade para o AMD MI300A — é executada separadamente no supercomputador Santos Dumont (LNCC/SINAPAD), acessado por meio do próprio programa de alocação do LNCC e fora do escopo desta solicitação.

Solicitação de Recursos

Recurso	Solicitação
Hardware físico	1× RTX PRO 6000 Blackwell — hardware de desenvolvimento local dedicado, utilizado ao longo de todo o projeto

O tempo de computação no Santos Dumont é solicitado separadamente por meio do próprio programa de alocação de projetos do LNCC (SINAPAD) e não faz parte desta solicitação de hardware.

Referências

- [1] Gabriel Fabien-Ouellet. Seismic modeling and inversion using half-precision floating-point numbers. *Geophysics*, 85(3):F65–F76, 2020.
- [2] Longfei Gao and Kevin Harms. Half precision wave simulation. arXiv:2310.00236, 2023. Revisado em 2024.
- [3] LSC-Unicamp. GPUZIP source code. <https://github.com/LSC-Unicamp/GPUZIP>, 2025. Acesso em 2026.

- [4] Thiago Maltempi, Sandro Rigo, Marcio Pereira, Hervé Yviquel, Gustavo Leite, Otavio Lee, Jesse Costa, and Guido Araujo. Checkpointing fine-tuning for accelerating seismic applications in GPUs. *International Journal of High Performance Computing Applications*, 39(6):784–802, 2025.
- [5] Takumi Uchino et al. Guaranteed DGEMM accuracy while using reduced precision tensor cores through extensions of the Ozaki scheme. arXiv:2511.13778, 2025. Apresentado no SCA/HPCAsia 2026, Osaka, Japão.